

Solución Parcial

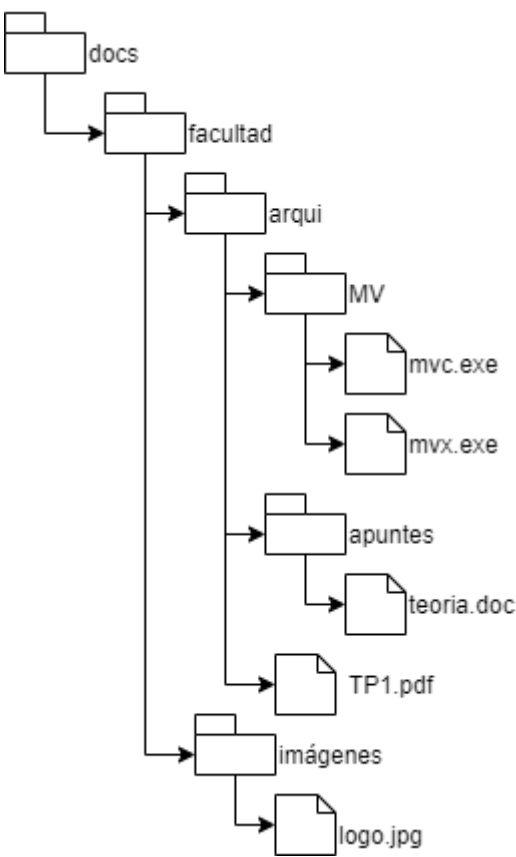
Visión general

Un sistema de archivos es representado por un árbol. Cada nodo del árbol puede ser un archivo o un directorio (carpeta). Los nodos directorios pueden tener hijos que pueden ser archivos o directorios.

Cada nodo del árbol a su vez tiene una estructura con los metadatos del archivo/directorio (Nombre, fecha de creación, fecha de modificación , tamaño, etc).

Cada archivo o directorio está asociado a un usuario llamado owner (el dueño del archivo) y a un grupo (group) al que pertenece el archivo.

Además el sistema de archivos tiene una lista de usuarios, donde cada usuario pertenece a 1 o más grupos. Y una lista de grupos donde cada grupo contiene a 1 o más usuarios. La información es coherente y simétrica, si un usuario pertenece a un grupo, ese grupo contendrá al usuario. No puede haber 2 usuarios con el mismo nombres, ni 2 grupos con el mismo nombre.

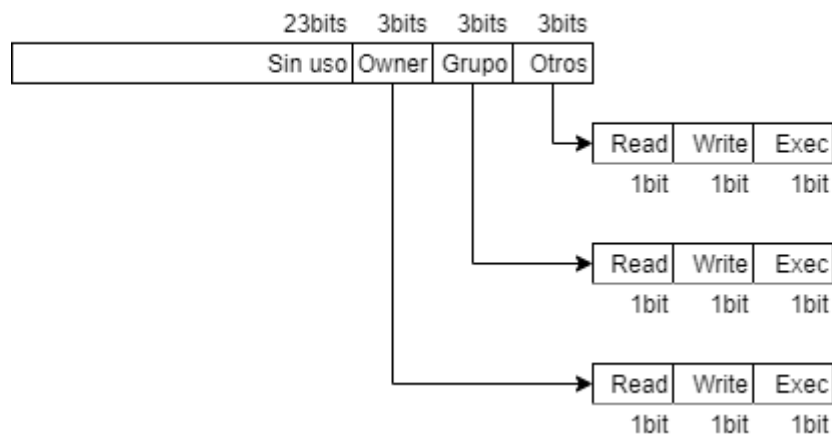


Nodo Arbol	Nodo Metadatos
Id Nombre Archivo/Directorio Puntero a Metadatos Permisos (opcional) Listahijos[]	Id (duplicado) Nombre (opcional) Puntero a nodo árbol Group_ID Owner_ID FechaCreacion HoraCreacion FechaModificacion HoraModificacion Permisos Tamaño (en bloques) ListaBloques[]

Nodo Usuario	Nodo Grupos
Id Nombre Grupos[]	Id Nombre Usuarios[]

Tipos de datos

Formato Permisos



Formato Fecha

2 Bytes 1 Byte 1 Byte

Año	Mes	Día
-----	-----	-----

Formato Hora

6bits 6 bits 6bits 14bits

Hora	Min	Seg	MSeg
------	-----	-----	------

Formato de fecha: 1 Celda de 4 bytes, en los cuales son, de más significativos a menos significativos: 2 bytes para el año, 1 byte para el mes (1 a 12) y 1 byte para el día (1 a 30).

Formato de hora: 1 celda de 32 bits, en los cuales son, de más significativo a menos significativos: 6 bits para la hora (0 a 23), 6 bits para los minutos (de 0 a 59), 6 bits los segundos (0 a 59) y 14 bits para milésimas de segundos (0 a 999).

Formato de Permisos:

Representan el acceso que puede tener un usuario a un archivo o directorio. Es un atributo propio del archivo o directorio (carpeta). Consta de 1 celda donde solo se utilizan los 9 bits menos significativos.

De más significativo a menos significativo son: 3 bits para el **owner**, 3 para **grupo** y 3 bits para **otros** usuarios.

A su vez cada permiso se subdivide en 1 bit para acceso de lectura, 1 bit para acceso escritura y 1 bit para acceso de ejecución.

Cuando el bit tiene valor 1 significa que el permiso está otorgado, cuando está en 0 el permiso está denegado.

Se recomienda el uso de sistema de numeración octal para trabajar con permisos.

Ejemplos:

- Si en permisos hay un **@777** (todos los bits en 1) quiere decir que tanto el owner, como el miembro del grupo, como cualquier usuario tiene permiso de lectura escritura y ejecución.
- Si en permisos hay un **@751** (binario 111 101 001) quiere decir que el owner puede leer, escribir y ejecutar; el miembro del grupo puede leer y ejecutar pero no escribir; y por último el resto de los usuarios pueden solo ejecutar.

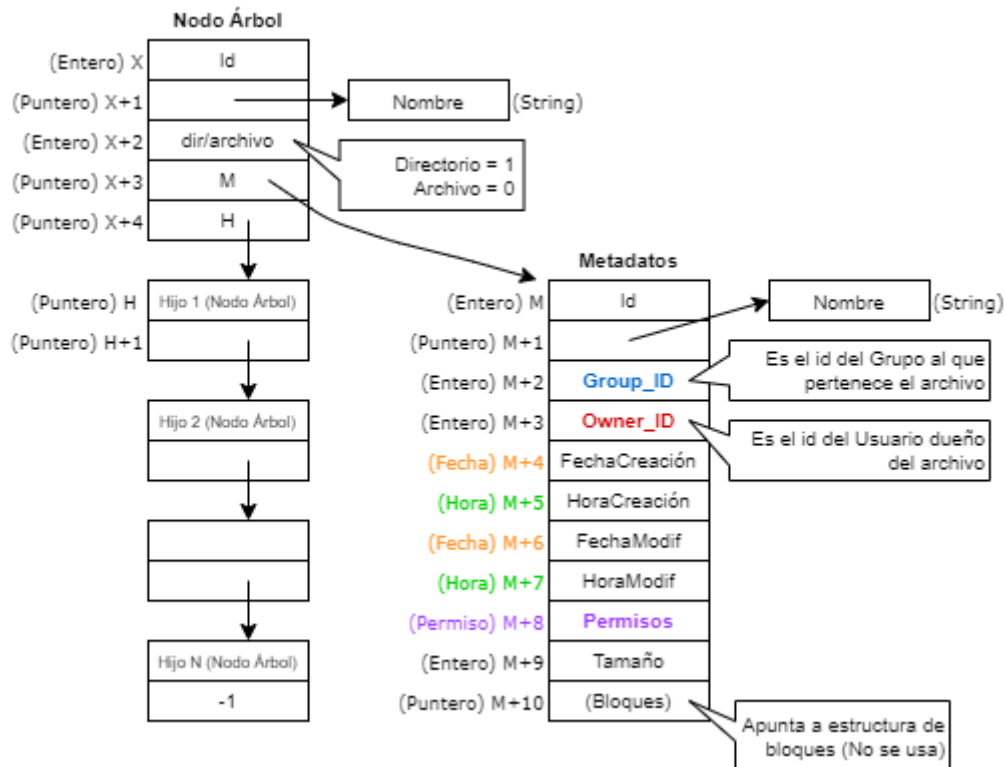
Estructura del árbol

Los archivos y directorios (carpetas) se representan con un nodo de un árbol en una dirección X de memoria. Se diferencia archivos de directorios por el campo en X+2 que si tiene un **1** el nodo corresponde a un **directorio** y si tiene un **0** corresponde a un **archivo**.

Cada nodo del árbol apunta en X+3 a una estructura de metadatos de los archivos/directorios que comienza en una celda M. En los metadatos del archivo se encuentra el **id del usuario** (entero positivo) que es dueño del archivo (OWNER_ID) en M+3 y el **id del grupo** (entero positivo) al que pertenece el archivo (GROUP_ID) en M+2.

Los campos tamaño (M+9) y bloques (M+10) **solo son válidos para los archivos**.

Si el nodo es un directorio, en X+4 **apunta a una lista de hijos**. Cada nodo de la lista tiene un puntero a nodo de árbol y un puntero al siguiente. **Si el directorio está vacío** en X+4 habrá un -1 (null).



Estructura de usuarios y grupos

La estructura de **usuarios** consta de un registro de **3 celdas** comenzando en la **celda V**:

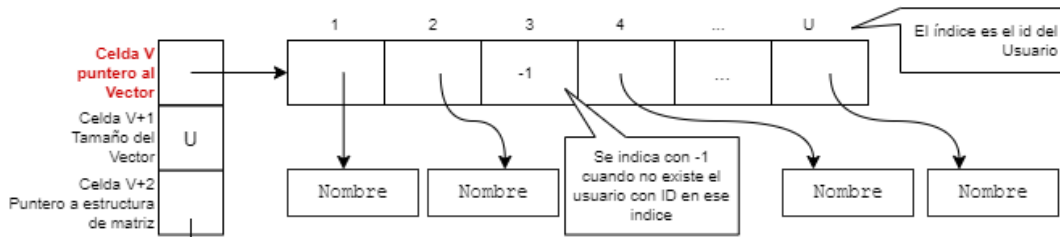
- V tiene un puntero a un vector de strings cuyo índice (comenzando en 1) es el id del usuario y el contenido de la celda un puntero al nombre.
- V+2 tiene el tamaño del vector en cantidad de celdas.
- V+3 apunta a una estructura que comienza en la celda T y vincula usuarios y grupos.

La estructura de **grupos**, es idéntica a la de usuarios, tiene **3 celdas** comenzando en la **celda Y**, la diferencia es que contiene los nombre de los grupos.

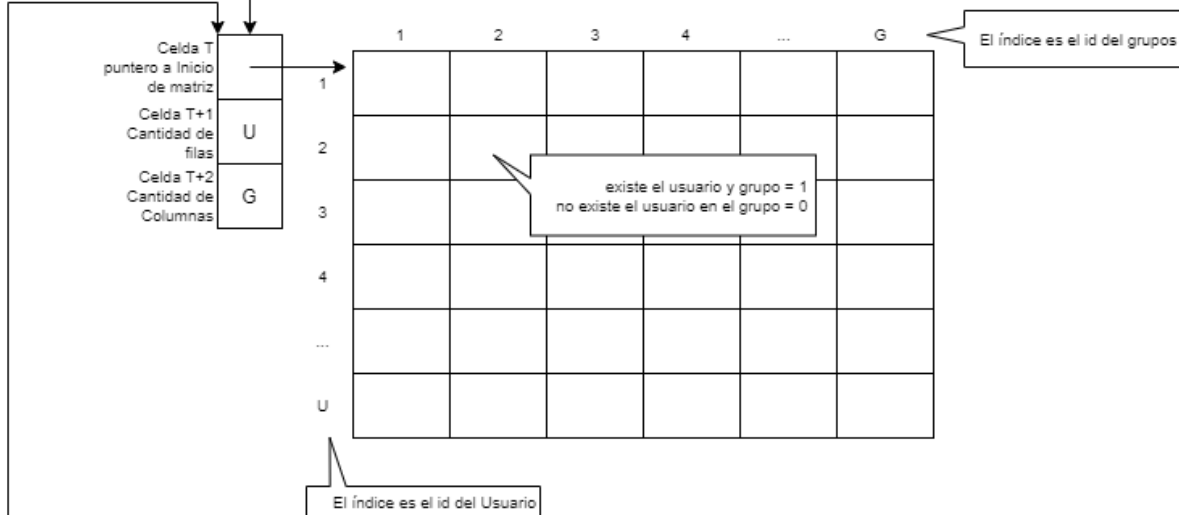
La estructura vincula usuarios y grupos consta de 3 celdas consecutivas comenzando en la celda T:

- En T hay un **puntero a una matriz** de Usuarios por Grupo que **se implementa por filas consecutivas en la memoria**.
- En T+1 está la **cantidad de columnas** (es decir la cantidad de grupos).
- En T+2 está la **cantidad de filas** (es decir la cantidad de usuarios).

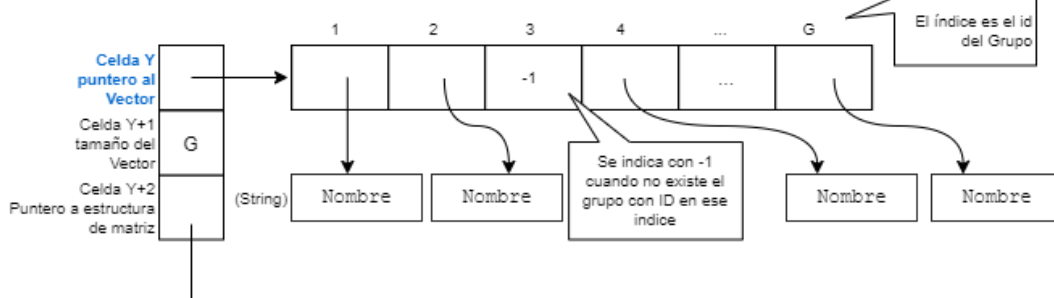
Estructura de Usuarios (Owner_ID)



Estructura de Usuarios por grupo



Estructura de Grupos (Group_ID)



Ejercicio

Escriba la (o las) subrutina(s) necesaria(s) para recibir un puntero a un nodo del árbol (X) de un directorio, la dirección de memoria (V) de la estructura usuarios, un puntero a nombre de usuario, una fecha y una hora. La subrutina debe **devolver en AX el tamaño ocupado en bloques de archivos modificados posteriormente a la fecha y hora y que el usuario pertenezca al grupo** del archivo pasado por parámetro.

La subrutina principal debe ser equivalente al siguiente protocolo en C:

```
int cantBloques (TArbol* directorio, TUsuarios* usuarios, char* nombre, int fecha,
int hora)
```

Considerar que **si no encuentra el usuario, debe devolver 0**.

Solución

En primer lugar debemos definir la invocación a la subrutina tal cual se especifica en el protocolo C.

```
PUSH <hora>
PUSH <fecha>
PUSH <puntero a nombre de usuario>
PUSH <puntero a estructura estructura de usuarios>
PUSH <puntero a nodo del árbol>
CALL CANTBLOQUES
ADD SP,5
Ax <- devuelve la cantidad de bloques
```

En segundo lugar debemos planear la ejecución del objetivo y ubicar dónde están los datos que necesitamos.

Hay que sumar los bloques -este dato se encuentra en el nodo del árbol- por lo tanto habrá que recorrer todos los nodos. El tamaño en bloques se encuentra en los metadatos de cada archivo.

En cada archivo del recorrido hay que validar, que sea posterior a la fecha y hora pasada por parámetro. La fecha y hora del archivo están en los metadatos.

Además hay que verificar que el usuario pertenezca al mismo grupo del archivo. El grupo del archivo es un atributo que se encuentra en los metadatos y el id de usuario se obtiene de la estructura de usuarios. Y finalmente con ambos ids, se debe ubicar la posición en la matriz y verificar que el usuario pertenezca al grupo.

Entonces, en este caso parece conveniente hacer que la subrutina **cantbloques** sea recursiva y que recorra cada archivo.

Para probar la idea bosquejamos una solución:

```
CANTBLOQUES:    PUSH    BP
                MOV     BP, SP
                PUSH    <registros utilizados, menos AX donde se devuelve el resultado>

                MOV     AC, 0 ;utilizo un registro auxiliar como acumulador

                CMP     <Archi/dir>, 0 ; ¿Es archivo?
                JZ      CANTBLOQUES_ARCHIVO

CANTBLOQUES_HIJO:
                ;si no es archivo, hay que hacer la invocación recursiva con los hijos
                CMP     ¿fin de lista?
                jz      CANTBLOQUES_FIN

                PUSH    <Parámetros>
                CALL    CANTBLOQUES
                ADD     SP, <Cant parametros>

                ADD     AC, AX ;Acumulo el resultado

                JMP     CANTBLOQUES_HIJO

CANTBLOQUES_ARCHIVO: ;Verificamos si el el archivo es posterior

                CMP     <FechaArchivo>, <FechaParámetro>
                JP      VERGRUPO
                JN      CANTBLOQUES_FIN
```

```

CMP     <HoraArchivo>, <HoraParámetro>
JP      VERGRUPO
JMP     CANTBLOQUES_FIN

```

VERGRUPO: ;Averiguar el usuario pertenece al grupo

```

PUSH    <Estructura, usuario y grupo>
CALL    USUARIOENGRUPO ; verifica si un usuario pertenece al grupo
ADD     SP, <3>
; Devuelve en AX TRUE (-1) si pertenece o FALSE (0) si no pertenece

```

```

CMP     AX, FALSE
JZ      CANTBLOQUES_FIN

```

```

MOV     AC, <Cantidad de bloques del archivo>

```

CANTBLOQUES_FIN:

```

MOV     AX, AC ;devuelvo el valor en AX
POP     <registros utilizados, menos AX>
MOV     SP, BP
POP     BP
RET

```

Teniendo el “esqueleto” de lo que sería la subrutina podemos tener una mejor idea de lo necesario.

En la subrutina USUARIOENGRUPO sería conveniente tener el id del usuario y no le nombre. Para obtener el id del usuario hay que hacer un recorrido en la estructura de usuarios. Sería ideal realizar esa búsqueda una sola vez. Entonces tendríamos que hacerla antes de entrar en la recursividad. Por lo tanto podemos volver a pensar la subrutina principal CANTBLOQUES como una subrutina de 2 pasos: 1) averiguar el id del usuario, 2) recorrer el árbol.

Para ello necesitaremos:

; Obtener idUsuario

```

PUSH <puntero a nombre de usuario> ; dir de memoria donde comienza el str
PUSH <puntero a estructura estructura de usuarios>
CALL GETUID
ADD SP,2
Ax <- devuelve id de usuario, 0 (cero) si no lo encuentra

```

; Validar si un usuario pertenece a un grupo

```

PUSH <id grupo>
PUSH <id usuario>
PUSH <puntero a estructura estructura de usuarios>
CALL ENGRUPO
ADD SP,3
Ax <- devuelve TRUE (-1) si pertenece, FALSE (0) si no.

```

; Recorrer el árbol y validar (recursiva)

```

PUSH <Hora>
PUSH <fecha>
PUSH <id de usuario>
PUSH <puntero a estructura estructura de usuarios>
PUSH <puntero a nodo del árbol>

```

```

CALL CUENTABLOQUES
ADD SP,5
Ax <- devuelve la cantidad de bloques

```

Este planteo mostrado, es una forma de pensarlo. El “esqueleto” podría ser un diagrama en un papel borrador, escribirlo en un archivo de texto o simplemente un esquema mental. Aquí solo se describió un modo de pensarlo.

Ahora bien con estos elementos si realizaremos la subrutina principal:

```

CANTBLOQUES:    PUSH    BP
                MOV     BP, SP

                PUSH    [BP+4]        ;puntero a nombre de usuario
                PUSH    [BP+3]        ;puntero a estructura estructura de usuarios
                CALL    GETUID
                ADD     SP,2
                ; Ax <- devuelve id de usuario, 0 (cero) si no encuentra el usuario

                CMP     AX, 0
                JZ      CANTBLOQUES_FIN
                ; acá coincide que justo AX devuelve 0 si no hay usuarios,
                ; y ese 0 se convierte en la cantidad de bloques
                ; pero si no coincide habría que hacer la conversión

                ; si AX no es cero, recorre el árbol

                PUSH    [BP+6]        ;Hora
                PUSH    [BP+5]        ;Fecha
                PUSH    AX            ;idUser devuelto por GETUID
                PUSH    [BP+3]        ;puntero a estructura estructura de usuarios
                PUSH    [BP+2]        ;puntero a nodo del árbol
                CALL    CUENTABLOQUES
                ADD     SP,5
                ; Ax <- devuelve la cantidad de bloques

CANTBLOQUES_FIN: MOV     SP, BP
                POP     BP
                RET

```

A continuación resolvemos GETUID:

```

null            equ     -1

GETUID:         PUSH    BP
                MOV     BP, SP
                PUSH    bx
                PUSH    dx
                PUSH    dx
                PUSH    fx

                ; La estructura de usuarios se compone de 3 celdas
                ; la primer celda tiene la dirección de memoria donde comienza el vector
                ; la segunda la cantidad de celdas del vector
                ; la tercera apunta a la estructura de usuarios x grupo (No se usa en esta subrutina)

                ; La posición de el nombre de usuario en el vector es el id (comenzando en 1)
                ; La idea es recorrer el vector contando las posiciones en AX

```

; utilizando a bx para recorrer las celdas y en cx un puntero al final

```
mov    bx, [bp+2] ;puntero a estructura estructura de usuarios (Celda V)
mov    cx, [bx+1] ;guarda la cantidad de usuarios (valor U)
mov    bx, [bx+0] ;asigna la posición inicial del vector de usuarios
add    cx, bx      ;asigna a cx la celda siguiente a la última del vector
mov    ax, 1       ;se inicializa la posición (ID) en 1
mov    dx, [bp+3] ;puntero a nombre de usuario
```

```
GETUID_SIG:    cmp    bx, cx
               jz     GETUID_NOENC ;Finaliza el recorrido

               cmp    [bx], null    ;verifica que haya un puntero válido
               jz     GETUID_INC

               mov     fx, [bx]
               scmp    [fx], [dx]    ;compara los nombres
               jz     GETUID_FIN      ;si encuentra ya tiene en AX el idUsuario

GETUID_INC:    add     ax, 1
               add     bx, 1
               jmp     GETUID_SIG
```

```
GETUID_NOENC:  mov     AX, 0          ;si no encuentra asigna 0 a AX
GETUID_FIN:    POP     fx
               POP     dx
               POP     cx
               POP     bx
               MOV     SP, BP
               POP     BP
               RET
```

Para resolver CUENTABLOQUES utilizamos el “esqueleto” original pensado para la subrutina principal:

```
true          equ     -1
false         equ     0
CUENTABLOQUES: PUSH    BP
               MOV     BP, SP
               PUSH    ac
               PUSH    bx
               PUSH    dx

               mov     ac, 0          ;registro auxiliar como acumulador de la cantidad
               mov     bx, [BP+2]     ;puntero a nodo del árbol

               cmp     bx, null
               jz     CUENTABLOQUES_FIN

               CMP     [bx+2], 0      ;¿Es archivo?
               JZ      CUENTABLOQUES_ARCH

               ;si no es archivo, hay que hacer la invocación recursiva con los hijos
               ;los hijos están implementados como una lista dinámica donde cada nodo
               ;se compone de 2 celdas: 1) puntero a nodo árbol, 2) puntero a siguiente nodo.
               mov     dx, [bx+4]     ;pongo en dx el puntero al primer hijo (si tiene)

CUENTABLOQUES_HIJO: CMP     dx, null    ;¿fin de lista?
                   jz     CUENTABLOQUES_FIN
```



```

; invocación recursiva, uso los mismos parámetros en el mismo orden
; a diferencia del puntero al nodo de árbol
PUSH    [BP+6]                ;Hora
PUSH    [BP+5]                ;Fecha
PUSH    [BP+4]                ;idUser devuelto por GETUID
PUSH    [BP+3]                ;puntero a estructura estructura de usuarios
PUSH    [dx+0]                ;puntero a nodo del árbol hijo
CALL    CUENTABLOQUES
ADD     SP,5                  ;Ax <- devuelve la cantidad de bloques
ADD     AC, AX                ;Acumulo el resultado

mov     dx, [dx+1]            ;se asigna el siguiente nodo
JMP     CUENTABLOQUES_HIJO

```

```

CUENTABLOQUES_ARCH:  mov     dx, [bx+3]                ;aquí dx tendrá los metadatos del archivo
                    cmp     [dx+6], [BP+5]            ;¿la fecha del archivo es posterior?
                    jp      VERGRUPO
                    jn      CUENTABLOQUES_FIN        ;si la fecha es menor, se va
;acá llega cuando las fechas son iguales
                    cmp     [dx+7], [BP+6]            ;¿la hora del archivo es posterior?
                    jp      VERGRUPO
                    JMP     CUENTABLOQUES_FIN        ;No es posterior => se va

```

```

VERGRUPO:            ;Averiguar el usuario pertenece al grupo
                    PUSH    [dx+2]                ;id grupo del archivo
                    PUSH    [bp+4]                ;id usuario pasado por parámetro
                    PUSH    [bp+3]                ;puntero a estructura estructura de usuarios
                    CALL    ENGRUPO
                    ADD     SP,3                  ;Ax <- devuelve TRUE (-1) si pertenece, FALSE (0) si no.

                    cmp     ax, false
                    JZ      CUENTABLOQUES_FIN

                    MOV     AC, [dx+9]            ;Cantidad de bloques del archivo

```

```

CUENTABLOQUES_FIN:  MOV     AX, AC ;devuelvo el valor en AX
                    POP     dx
                    POP     bx
                    POP     ac
                    MOV     SP, BP
                    POP     BP
                    RET

```

Finalmente solo queda resolver la subrutina ENGRUPO que devuelve TRUE o FALSE dependiendo si el usuario pertenece al grupo:

```

ENGRUPO:            PUSH    BP
                    MOV     BP, SP
                    PUSH    bx
                    PUSH    dx

```

```

;La relación entre usuarios y grupos en esta implementación está hecha con una matriz
;por lo tanto tenemos que encontrar la celda (usuario,grupo) y verificar que haya un 1
;recibimos como parámetro la estructura de usuarios
;en la 3er celda de la estructura de usuarios, está la estructura de matriz
;la estructura de matriz también tiene 3 celdas:
;    +0 el puntero a la primer celda de la matriz (propiamente dicha)

```

```

;      +1 cantidad de usuarios (filas)
;      +2 cantidad de grupos (columnas)

mov     ax, false           ;por defecto devuelve false
mov     bx, [bp+2]          ;puntero a estructura de usuarios
mov     bx, [bx+2]          ;puntero a estructura de matriz

cmp     [bx+1], [bp+3]      ;veo que el id de usuario esté dentro de la matriz
jn      ENGRUPO_FIN
cmp     [bx+2], [bp+4]      ;veo que el id de grupo esté dentro de la matriz
jn      ENGRUPO_FIN

; en dx se calculará la celda de la matrix
mov     dx, [bp+3]          ;partimos de la posición del id usuario
sub     dx, 1               ;resta 1, para ajustar el offset (la matriz comienza en 0)
                                ;cada fila tiene G celdas (cantidad de columnas)
mul     dx, [bx+2]          ;=>multiplica por la cantidad de columnas
add     dx, [bp+4]          ;suma el id grupo
sub     dx, 1               ;ajusta la posición
; hasta acá se tiene en dx la posición de la celda de la matriz, relativa al comienzo
add     dx, [bx]            ;=>se le suma la posición de comienzo de la matriz

cmp     [dx], 0
jz      ENGRUPO_FIN         ;si es 0, no se hace nada (devuelve false)
mov     ax, true            ;si es 1 (no 0) devuelve true

ENGRUPO_FIN:
POP     dx
POP     bx
MOV     SP, BP
POP     BP
RET

```